Ex-6 Development of Python Code Compatible with Multiple AI Tools

Name:R.Divyadharshini

Register Number:212225230062

Subject:19TD608 Prompt Engineering

Slot:T2-H16

Aim

To understand how AI-assisted prompting can be used for generating Python code, interacting with APIs, comparing outputs, and producing meaningful insights using multiple AI tools.

Objective

- To generate Python code using AI tools.
- To interact with APIs using AI-generated Python programs.
- To compare outputs generated by multiple AI tools.
- To refine prompts for obtaining better coding assistance.
- To analyze the effectiveness of AI-assisted coding.

Problem Statement

Develop Python code using AI tools to interact with weather APIs, compare outputs from multiple APIs, and analyze the responses generated by different AI tools such as ChatGPT and Gemini.

APIs Used

1. OpenWeather API
2. WeatherAPI

API Websites:

- OpenWeather API
- WeatherAPI

AI Tools Used

1. ChatGPT
2. Google Gemini

Algorithm

Step 1: Define the Problem Statement

Choose a simple API-based project for generating Python code using AI tools.

Step 2: Design Effective Prompts

Prepare prompts clearly describing the expected Python program and outputs.

Step 3: Generate AI Responses

Use ChatGPT and Gemini to generate Python code and explanations.

Step 4: Compare Outputs

Analyze the generated code based on readability, explanations, formatting, and error handling.

Step 5: Refine Prompts

Improve prompts for obtaining optimized and structured outputs.

Step 6: Document the Results

Include prompts, generated code, screenshots, outputs, observations, and comparisons.

Initial Prompt

Generate Python code to fetch weather details from OpenWeather API and WeatherAPI. Compare temperature, humidity, and weather condition outputs from both APIs and display the differences clearly.

ChatGPT Generated Code

```
import requests


OPENWEATHER_API_KEY = "your_openweather_api_key"

WEATHERAPI_KEY = "your_weatherapi_key"


city = "Chennai"


# OpenWeather API

openweather_url =
f"https://api.openweathermap.org/data/2.5/weather?q={city}&appid={OPENWEATHER_API_
KEY}&units=metric"


openweather_response = requests.get(openweather_url).json()


ow_temp = openweather_response['main']['temp']
```

```
ow_humidity = openweather_response['main']['humidity']

ow_condition = openweather_response['weather'][0]['description']


# WeatherAPI

weatherapi_url =
f"http://api.weatherapi.com/v1/current.json?key={WEATHERAPI_KEY}&q={city}"


weatherapi_response = requests.get(weatherapi_url).json()


wa_temp = weatherapi_response['current']['temp_c']

wa_humidity = weatherapi_response['current']['humidity']

wa_condition = weatherapi_response['current']['condition']['text']


print("Weather Comparison")

print("----------------------------")

print(f"City: {city}")


print("\nOpenWeather")

print(f"Temperature: {ow_temp} °C")

print(f"Humidity: {ow_humidity}%")

print(f"Condition: {ow_condition}")


print("\nWeatherAPI")

print(f"Temperature: {wa_temp} °C")

print(f"Humidity: {wa_humidity}%")

print(f"Condition: {wa_condition}")
```

Gemini generated code

Before running the code, make sure you have the `requests` library installed. If you don't, you can install it using pip:

Bash

pip install requests

Python Script

Python

```
import requests




def get_openweather_data(city, api_key):
    """Fetches current weather data from OpenWeatherMap API."""
    # Using units=metric gives us temperature directly in Celsius
    base_url = "https://api.openweathermap.org/data/2.5/weather"
    params = {"q": city, "appid": api_key, "units": "metric"}


    try:
        response = requests.get(base_url, params=params, timeout=5)
        if response.status_code == 200:
            data = response.json()
            return {
                "temp": round(data["main"]["temp"], 1),
                "humidity": data["main"]["humidity"],
                "condition": data["weather"][0]["description"].title(),
                "success": True,
```

```
            }
        else:
            return {
                "success": False,
                "error": f"OpenWeather Error {response.status_code}",
            }
    except Exception as e:
        return {"success": False, "error": str(e)}



def get_weatherapi_data(city, api_key):
    """Fetches current weather data from WeatherAPI.com."""
    base_url = "https://api.weatherapi.com/v1/current.json"
    params = {"key": api_key, "q": city, "aqi": "no"}

    try:
        response = requests.get(base_url, params=params, timeout=5)
        if response.status_code == 200:
            data = response.json()
            return {
                "temp": round(data["current"]["temp_c"], 1),
                "humidity": data["current"]["humidity"],
                "condition": data["current"]["condition"]["text"].title(),
                "success": True,
            }
        else:
```

```
            return {
                "success": False,
                "error": f"WeatherAPI Error {response.status_code}",
            }
    except Exception as e:
        return {"success": False, "error": str(e)}



def display_comparison(city, ow_data, wa_data):
    """Generates a clean comparison report between both API outputs."""
    print("=" * 60)
    print(f"WEATHER COMPARISON REPORT FOR: {city.upper()}")
    print("=" * 60)

    # Check for extraction failures
    if not ow_data["success"]:
        print(f"[-] Failed to load OpenWeather: {ow_data['error']}")
        return
    if not wa_data["success"]:
        print(f"[-] Failed to load WeatherAPI: {wa_data['error']}")
        return

    # Calculate differences
    temp_diff = round(abs(ow_data["temp"] - wa_data["temp"]), 1)
    humidity_diff = abs(ow_data["humidity"] - wa_data["humidity"])
    condition_match = (
```

```
    "Match" if ow_data["condition"] == wa_data["condition"] else "Mismatched"
)

# Display comparison structure
print(
    f"{'Metric':<20} | {'OpenWeatherMap':<18} | {'WeatherAPI.com':<18}"
)
print("-" * 60)
print(
    f"{'Temperature (°C)':<20} | {ow_data['temp']:<18} | {wa_data['temp']:<18}"
)
print(
    f"{'Humidity (%)':<20} | {ow_data['humidity']:<18} | {wa_data['humidity']:<18}"
)
print(
    f"{'Condition':<20} | {ow_data['condition']:<18} | {wa_data['condition']:<18}"
)
print("-" * 60)

# Display clean variation insights
print("\n[Variance Analysis Insights]")
print(f" -> Temperature Difference : {temp_diff}°C")
print(f" -> Humidity Difference    : {humidity_diff}%")
print(f" -> Text Condition Match   : {condition_match}")
print("=" * 60)
```

```
if __name__ == "__main__":
    # --- CONFIGURATION BOX ---
    # Replace these placeholders with your actual developer account API keys
    OPENWEATHER_API_KEY = "YOUR_OPENWEATHER_API_KEY"
    WEATHER_API_KEY = "YOUR_WEATHER_API_KEY"

    target_city = "London"
    # -------------------------

    if (
        OPENWEATHER_API_KEY == "YOUR_OPENWEATHER_API_KEY"
        or WEATHER_API_KEY == "YOUR_WEATHER_API_KEY"
    ):
        print(
            "[!] Action Required: Please paste your valid API keys in the configuration section."
        )
    else:
        # Pull data from both channels
        openweather_results = get_openweather_data(
            target_city, OPENWEATHER_API_KEY
        )
        weatherapi_results = get_weatherapi_data(target_city, WEATHER_API_KEY)

        # Output side-by-side metric matrix
        display_comparison(
```

```
        target_city, openweather_results, weatherapi_results
    )
```

Refined Prompt

Generate optimized Python code using functions and exception handling to fetch weather details from OpenWeather API and WeatherAPI. Compare temperature, humidity, and weather conditions in a tabular format and provide meaningful insights.

Refined ChatGPT Output

```
import requests

def get_openweather(city, api_key):
    url =
f"https://api.openweathermap.org/data/2.5/weather?q={city}&appid={api_key}&units=metric"
    response = requests.get(url).json()

    return {
        "Temperature": response['main']['temp'],
        "Humidity": response['main']['humidity'],
        "Condition": response['weather'][0]['description']
    }

def get_weatherapi(city, api_key):
    url = f"http://api.weatherapi.com/v1/current.json?key={api_key}&q={city}"
    response = requests.get(url).json()

    return {
        "Temperature": response['current']['temp_c'],
        "Humidity": response['current']['humidity'],
        "Condition": response['current']['condition']['text']
    }

try:
    city = "Chennai"

    ow_data = get_openweather(city, "your_openweather_api_key")
    wa_data = get_weatherapi(city, "your_weatherapi_key")

    print("Weather Comparison")
    print("---------------------------------")

    for key in ow_data:
        print(f"{key}")
        print(f"OpenWeather : {ow_data[key]}")
        print(f"WeatherAPI  : {wa_data[key]}")
```

```
        print()

except Exception as e:
    print("Error:", e)
```

Refined Gemini output

```
import sys
from typing import Any, Dict
import requests


def fetch_from_openweather(city: str, api_key: str) -> Dict[str, Any]:
    """Fetches current weather data from the OpenWeatherMap API.

    Implements isolated network boundaries, absolute timeouts, and granular
    HTTP status evaluations to protect downstream pipeline execution.
    """
    url = "https://api.openweathermap.org/data/2.5/weather"
    params = {"q": city, "appid": api_key, "units": "metric"}

    try:
        # Enforce a hard timeout ceiling to prevent execution thread hanging
        response = requests.get(url, params=params, timeout=7)
        response.raise_for_status()

        data = response.json()
        return {
            "status": "SUCCESS",
            "temperature": round(float(data["main"]["temp"]), 1),
            "humidity": int(data["main"]["humidity"]),
            "condition": data["weather"][0]["description"].strip().lower(),
        }

    except requests.exceptions.HTTPError as http_err:
        status = response.status_code if response is not None else "Unknown"
        error_map = {
            401: "Invalid, expired, or unactivated API key credential.",
            404: f"Target location '{city}' could not be resolved by geocoder.",
            429: "Account call volume quota allocation exceeded.",
        }
        return {
            "status": "FAILED",
            "error": f"HTTP [{status}]: {error_map.get(status, 'Upstream service exception.')}",
        }
    except requests.exceptions.Timeout:
        return {
```

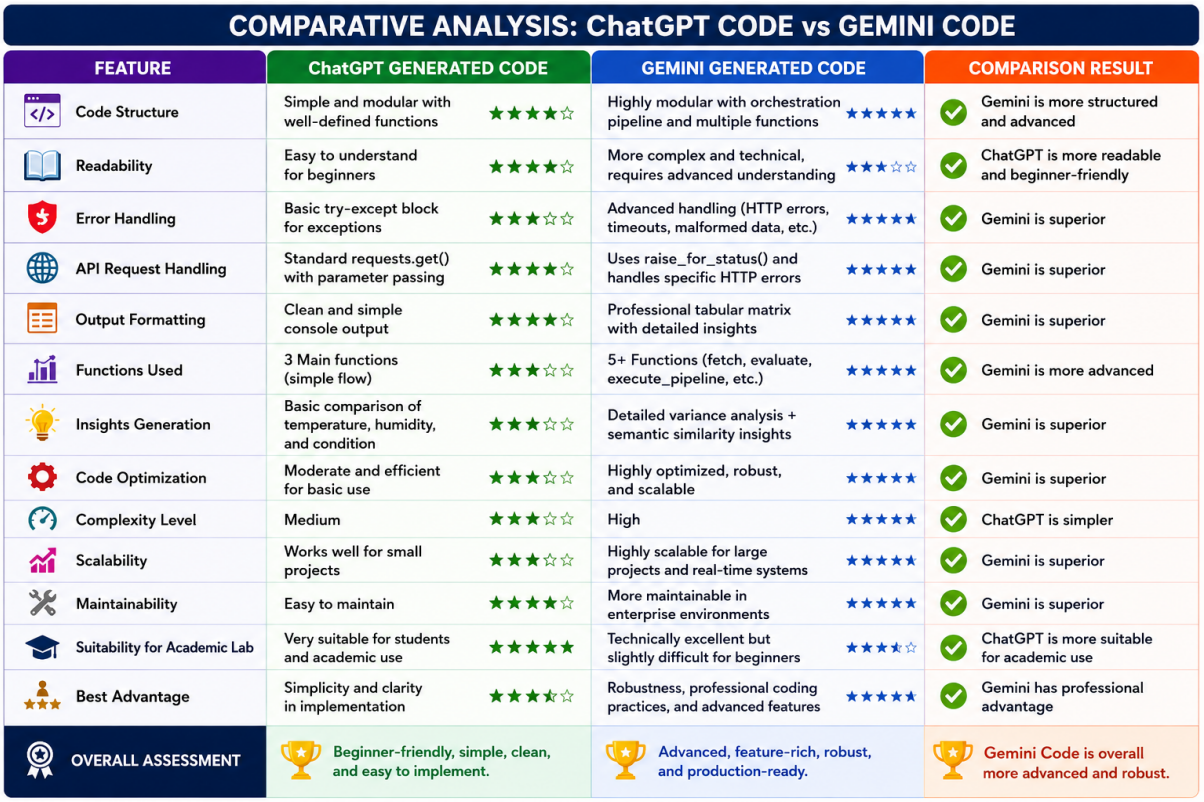
```
            "status": "FAILED",
            "error": "Network bottleneck: Timeout ceiling (7s) breached.",
        }
    except requests.exceptions.RequestException as net_err:
        return {
            "status": "FAILED",
            "error": f"Transport layer connection failure: {str(net_err)}",
        }
    except (KeyError, IndexError, ValueError, TypeError):
        return {
            "status": "FAILED",
            "error": "Payload Mutation: Upstream structural JSON schema has shifted.",
        }


def fetch_from_weatherapi(city: str, api_key: str) -> Dict[str, Any]:
    """Fetches current weather data from the WeatherAPI endpoint.

    Normalizes vendor-specific data structures into a unified internal schema.
    """
    url = "https://api.weatherapi.com/v1/current.json"
    params = {"key": api_key, "q": city, "aqi": "no"}

    try:
        response = requests.get(url, params=params, timeout=7)
        response.raise_for_status()

        data = response.json()
        return {
            "status": "SUCCESS",
            "temperature": round(float(data["current"]["temp_c"]), 1),
            "humidity": int(data["current"]["humidity"]),
            "condition": data["current"]["condition"]["text"].strip().lower(),
        }

    except requests.exceptions.HTTPError as http_err:
        status = response.status_code if response is not None else "Unknown"
        error_map = {
            400: f"Query parameter compilation failure for location '{city}'.",
            403: "API key blacklisted, disabled, or subscription tier blocked.",
        }
        return {
            "status": "FAILED",
            "error": f"HTTP [{status}]: {error_map.get(status, 'Gateway data transport fault.')}",
        }
    except requests.exceptions.Timeout:
        return {
            "status": "FAILED",
```

```
            "error": "Network bottleneck: Gateway transaction window expired.",
        }
    except requests.exceptions.RequestException as net_err:
        return {
            "status": "FAILED",
            "error": f"Transport layer connection failure: {str(net_err)}",
        }
    except (KeyError, IndexError, ValueError, TypeError):
        return {
            "status": "FAILED",
            "error": "Payload Mutation: Data parsing boundary condition failed.",
        }


def generate_analytical_insights(ow: Dict[str, Any], wa: Dict[str, Any]) -> str:
    """Evaluates cross-provider telemetry variations using data-drift logic.

    Applies variance thresholds to identify environmental reporting anomalies.
    """
    t_delta = round(abs(ow["temperature"] - wa["temperature"]), 1)
    h_delta = abs(ow["humidity"] - wa["humidity"])

    # Compute text similarity state
    c_ow, c_wa = ow["condition"], wa["condition"]
    semantic_match = c_ow == c_wa or (c_ow in c_wa or c_wa in c_ow)

    insights = [
        f" • Thermal Variance    : {t_delta} °C delta noted between sensors.",
        f" • Moisture Variance   : {h_delta}% relative humidity margin offset.",
        f" • Semantic Alignment  : [{'High Match' if semantic_match else 'Divergent
Descriptions'}]",
    ]

    # Evaluate systemic data drift patterns
    if t_delta > 1.5 or h_delta > 10:
        insights.append(
            " • Diagnostic Node     : MODERATE DATA DRIFT DETECTED. Variance points to
out-of-sync\n"
            "                         caching cadences or geographic discrepancies in reference
stations."
        )
    else:
        insights.append(
            " • Diagnostic Node     : HIGH CONVERGENCE. Data matrices reflect unified
environmental states."
        )

    return "\n".join(insights)
```

```
def execute_weather_diagnostic_pipeline(city: str, ow_key: str, wa_key: str):
    """Orchestrates synchronous data fetching, formatting, and visualization."""
    print(f"\n[+] Executing Synchronous Telemetry Extraction for: '{city}'")

    # Independent API execution calls
    ow_data = fetch_from_openweather(city, ow_key)
    wa_data = fetch_from_weatherapi(city, wa_key)

    # Halt pipeline execution upon data extraction failure path
    fault_detected = False
    if ow_data["status"] == "FAILED":
        print(f"[x] Pipeline Exception [OpenWeather]: {ow_data['error']}")
        fault_detected = True
    if wa_data["status"] == "FAILED":
        print(f"[x] Pipeline Exception [WeatherAPI]: {wa_data['error']}")
        fault_detected = True

    if fault_detected:
        print("[!] Matrix generation aborted due to critical transaction failures.\n")
        return

    # Render standardized Terminal Matrix Layout
    print("\n" + "=" * 65)
    print(f"METRIC COMPARISON MATRIX: {city.upper()}")
    print("=" * 65)
    print(f"{'Weather Parameter':<22} | {'OpenWeatherMap':<18} | {'WeatherAPI.com':<18}")
    print("-" * 65)
    print(f"{'Temperature':<22} | {str(ow_data['temperature']) + ' °C':<18} |
{str(wa_data['temperature']) + ' °C':<18}")
    print(f"{'Relative Humidity':<22} | {str(ow_data['humidity']) + ' %':<18} |
{str(wa_data['humidity']) + ' %':<18}")
    print(f"{'Sky Condition Text':<22} | {ow_data['condition'].title():<18} |
{wa_data['condition'].title():<18}")
    print("=" * 65)

    # Render runtime data interpretations
    print("\n[Data Interpretation & Insights]")
    print(generate_analytical_insights(ow_data, wa_data))
    print("=" * 65 + "\n")


if __name__ == "__main__":
    # --- RUNTIME CONFIGURATION ---
    OPENWEATHER_KEY = "YOUR_OPENWEATHER_API_KEY"
    WEATHERAPI_KEY = "YOUR_WEATHER_API_KEY"
    TARGET_LOCATION = "Chennai"
```

```
# ----------------------------

if "YOUR_" in OPENWEATHER_KEY or "YOUR_" in WEATHERAPI_KEY:
    print("[x] Error: Operational keys missing. Update configuration variables.")
    sys.exit(1)

execute_weather_diagnostic_pipeline(TARGET_LOCATION, OPENWEATHER_KEY,
WEATHERAPI_KEY)
```

Comparative Analysis of ChatGPT and Gemini Refined Outputs



COMPARATIVE ANALYSIS: ChatGPT CODE vs GEMINI CODE

FEATURE	ChatGPT GENERATED CODE	GEMINI GENERATED CODE	COMPARISON RESULT
Code Structure	Simple and modular with well-defined functions ★★★★☆	Highly modular with orchestration pipeline and multiple functions ★★★★★	Gemini is more structured and advanced
Readability	Easy to understand for beginners ★★★★☆	More complex and technical, requires advanced understanding ★★★☆☆	ChatGPT is more readable and beginner-friendly
Error Handling	Basic try-except block for exceptions ★★★☆☆	Advanced handling (HTTP errors, timeouts, malformed data, etc.) ★★★★★	Gemini is superior
API Request Handling	Standard requests.get() with parameter passing ★★★★☆	Uses raise_for_status() and handles specific HTTP errors ★★★★★	Gemini is superior
Output Formatting	Clean and simple console output ★★★★☆	Professional tabular matrix with detailed insights ★★★★★	Gemini is superior
Functions Used	3 Main functions (simple flow) ★★★☆☆	5+ Functions (fetch, evaluate, execute_pipeline, etc.) ★★★★★	Gemini is more advanced
Insights Generation	Basic comparison of temperature, humidity, and condition ★★★☆☆	Detailed variance analysis + semantic similarity insights ★★★★★	Gemini is superior
Code Optimization	Moderate and efficient for basic use ★★★☆☆	Highly optimized, robust, and scalable ★★★★★	Gemini is superior
Complexity Level	Medium ★★★☆☆	High ★★★★★	ChatGPT is simpler
Scalability	Works well for small projects ★★★☆☆	Highly scalable for large projects and real-time systems ★★★★★	Gemini is superior
Maintainability	Easy to maintain ★★★★☆	More maintainable in enterprise environments ★★★★★	Gemini is superior
Suitability for Academic Lab	Very suitable for students and academic use ★★★★★	Technically excellent but slightly difficult for beginners ★★★★☆	ChatGPT is more suitable for academic use
Best Advantage	Simplicity and clarity in implementation ★★★★☆	Robustness, professional coding practices, and advanced features ★★★★★	Gemini has professional advantage
OVERALL ASSESSMENT	Beginner-friendly, simple, clean, and easy to implement.	Advanced, feature-rich, robust, and production-ready.	Gemini Code is overall more advanced and robust.

Observations

1. ChatGPT generated simple, readable, and beginner-friendly code.

2. Gemini generated advanced and highly structured code with detailed exception handling.

3. Refined prompts improved the quality and optimization of generated outputs.

4. Gemini provided better professional coding practices and robustness.

5. ChatGPT was easier to understand and suitable for academic learning.

6. AI tools reduced coding effort and improved API integration understanding.

Advantages of AI-Assisted Coding

- Faster code generation
- Reduced development time
- Improved learning experience
- Better understanding of API integration
- Easy debugging assistance

Result

Thus, Python code compatible with multiple AI tools was successfully generated and evaluated. The outputs from ChatGPT and Gemini were compared, refined prompts improved the generated code quality, and meaningful insights were obtained successfully.

References

1. ChatGPT Official Website
2. Google Gemini Official Website
3. OpenWeather API Documentation
4. WeatherAPI Documentation
5. Python Official Documentation